ATLANTECH

BACKEND IMPLEMENTATION PLAN

Building the Laravel backend in seven phases

Port API and Central SaaS API · tasks, dependencies and the contract with the AI team

7	20	106	~92.75
PHASES	WORK PACKAGES	TASKS	EFFORT DAYS

CONTENTS

1. Scope and how to read
2. Phases at a glance
3. Dependency map
4. Integration with the AI team
5. Phase 1 — Foundation and contract
6. Phase 2 — Access control and master data
7. Phase 3 — Requests, BAPLIE and the AI bridge
8. Phase 4 — The core loop
9. Phase 5 — Dispatch and field execution
10. Phase 6 — Yard, port licence and real AI
11. Phase 7 — Central SaaS
12. Post-MVP backlog and open inputs
13. Appendix — AI dependency coverage

SCOPE

The backend team builds the Laravel port API and the Central SaaS API. The Vue dashboard and the Flutter app have their own plans and consume the OpenAPI file this team keeps. The four Python AI workers belong to the AI team; the backend talks to them only through Redis queues and the port database.

HOW TO READ THE TABLES

Column	Meaning
ID	`P<phase>-<package><n>`, for example P3-I2
Effort	Days for one developer, tests included
Depends on	Tasks merged first; same phase unless prefixed
AI link	AI team dependency (AIBD), task (AI) or integration decision (D)
BLOCKED	Waits for an integration decision with the AI team

OVERVIEW

Phases at a glance

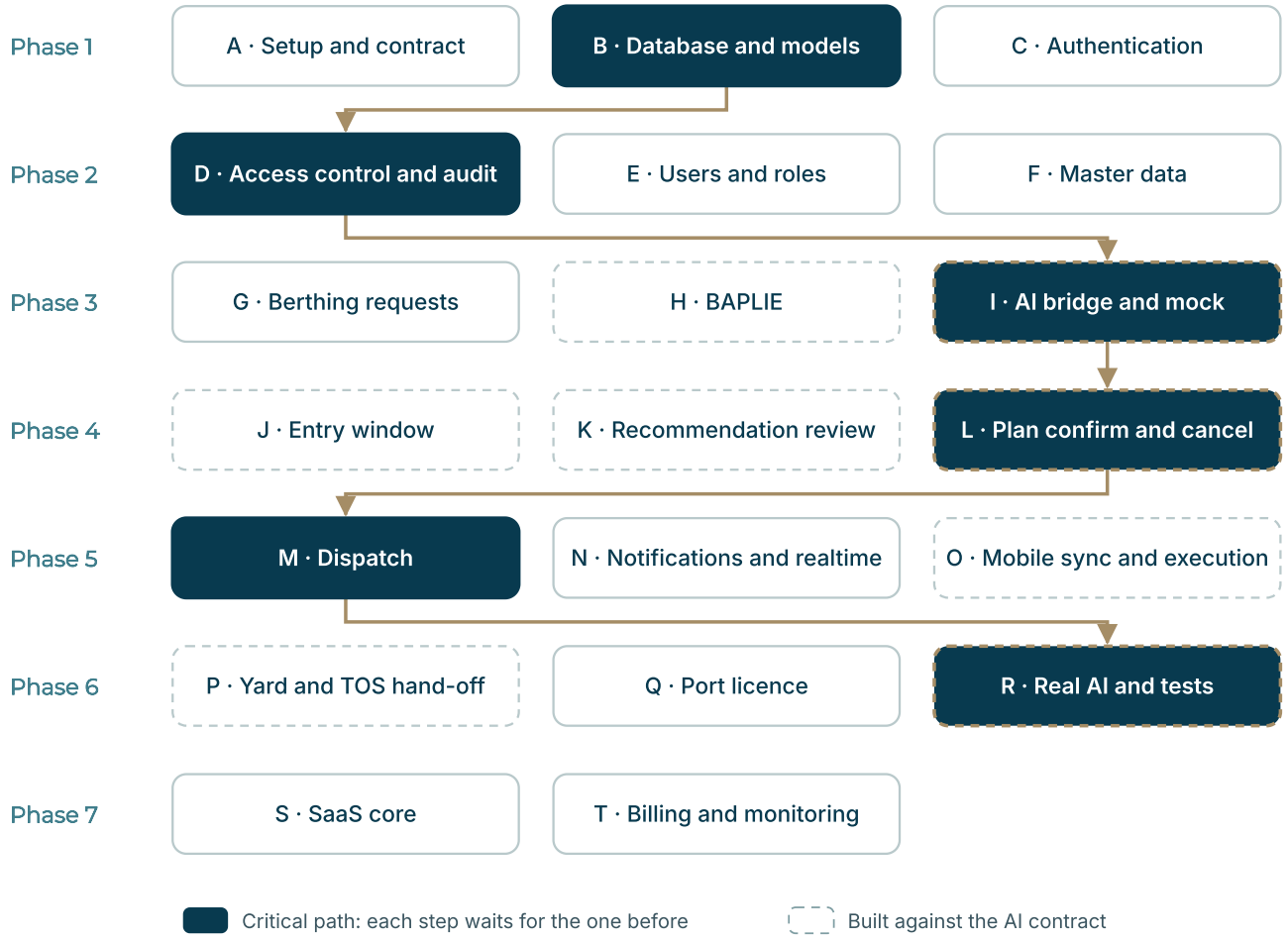
Phases 1 to 6 deliver the port-side MVP; Phase 7 adds the Central SaaS. The plan holds 106 tasks and about 92.75 effort days.

	Phase	Work packages	Effort
1	**Foundation and contract** One repository, every table migrated from the decided schema, sign-in working, and contract v1 agreed with the AI team.	A · Setup and contract B · Database and models C · Authentication	15.25
2	**Access control and master data** Every later endpoint inherits the same write chain, and the port's master data is complete enough for the AI to read.	D · Access control and audit E · Users and roles F · Master data	16.25
3	**Requests, BAPLIE and the AI bridge** The first AI-triggering flows exist and a mock worker answers them through the agreed contract. Work package I gates Phases 4 to 6.	G · Berthing requests H · BAPLIE I · AI bridge and mock	15.25
4	**The core loop** A request goes from customs approval to a confirmed plan. J, K and L run in parallel from seeded rows and the mock, then join at the end of the phase.	J · Entry window K · Recommendation review L · Plan confirm and cancel	13.25
5	**Dispatch and field execution** A confirmed plan reaches the pilot and the tug master, and field events come back as actual status and deviations.	M · Dispatch N · Notifications and realtime O · Mobile sync and execution	13
6	**Yard, port licence and real AI** The last port-side features land and the mock is swapped for each real worker the AI team delivers, EDI Parser first.	P · Yard and TOS hand-off Q · Port licence R · Real AI and tests	11.5
7	**Central SaaS** The central SaaS runs its own logic, so a port checks in against a real licence.	S · SaaS core T · Billing and monitoring	8.25

OVERVIEW

Dependency map

Six work packages form the critical path: a slip there moves every later phase. Everything else can slip inside its phase.



CONTRACT

Integration with the AI team

Eleven decisions are settled with the AI team in one joint session (P1-A5). The backend position is where the discussion starts, not a decision.

ID	Question	AI team recommends	Backend position	Blocks	Before
D-01	How Laravel learns a job finished (OD-01)	Completion message on queue:ai-results, plus a poller	Agree; the consumer runs as a supervised artisan command	P3-I3	Phase 3
D-02	Who writes request transitions, history, audit and run rows (OD-02)	Backend writes all; AI writes run progress and result tables	Agree	P3-I2, P4-K1	Phase 3
D-03	Where override, yard adjustment, BAPLIE correction and window re-check run (OD-03)	Async validation job (C-05)	Override and yard adjustment synchronous in Laravel with shared test vectors; BAPLIE correction async; window re-check in Laravel	P3-H6, P4-K4, P6-P3	Phase 3
D-04	Job envelope, versioning, retries (OD-22)	JSON envelope with RPUSH, contract_version 1.0	Agree; add the exact unprefixed key names	P3-I1	Phase 3
D-05	Contract v1: payloads C-01 to C-06 and JSON Schemas (AI-003)	Approve the proposed payloads	Approve once D-01 to D-04 are settled	P3-I1, P3-I8	Phase 3
D-06	What feeds the SSE endpoint	Not covered	Laravel streamed endpoint reading a short-lived events store; Reverb only for the mobile WebSocket	P3-I4	Phase 3
D-07	Which requests a solve may change, per trigger (OD-08)	Confirmed plans fixed; pending and trigger requests variable	Agree; it defines which pending versions become REPLACED	P4-K1, P4-K5	Phase 4
D-08	How NO_FEASIBLE_WINDOW is stored (OD-10)	Proposal with no windows, linked from the run	Agree; schema note change only	P4-J1	Phase 4
D-09	Read source and database privileges (OD-21)	Primary for reads; one account per worker	Agree; the backend writes the grants script	P3-I5	Phase 3
D-10	Where deviation thresholds live (OD-16)	port_setting keys per component type	Agree	P5-O6	Phase 5
D-11	Do weather pulls and Part 3 run in READ_ONLY (OD-18)	Yes, no user-visible writes	Agree	P6-Q1	Phase 6

BACKEND NOTES ON THE CONTRACT

Redis key prefix

Laravel prefixes every Redis key by default, so a job pushed to `queue:part1` would land under another key. AI jobs go through a connection with no prefix, and contract v1 names the exact keys.

SSE is not Reverb

Reverb serves WebSocket only. The dashboard's SSE stream is a Laravel streamed endpoint fed by an event source (D-06).

solver_run.solver_version

The column is NOT NULL but only the worker knows the value. The backend writes a placeholder from `port_setting` when it creates the row, and the worker overwrites it.

Redis down at commit

The run row is created in the same transaction as the status change, the job is pushed after commit, and a sweeper re-enqueues runs that stay QUEUED.

Worker delivery dates

Phase 6 swaps the mock for each real worker. The AI team gives a target per worker (EDI, Part 1, Part 2, Part 3); the mock stays as the fallback.

PHASE 1

Foundation and contract

17 tasks · 15.25 days

GOAL

One repository, every table migrated from the decided schema, sign-in working, and contract v1 agreed with the AI team.

EXIT CHECK

migrate:fresh --seed runs clean on an empty database; web and mobile sign-in tests pass; the contract v1 decision record is signed by both leads.

A Setup and contract

5 tasks · 4 days

P1-A1	project-backend repository with one folder per domain module; Docker Compose with PHP, NGINX, MySQL 8 and Redis	1		
P1-A2	API base: / `api/v1`, RFC 9457 problem responses with stable codes (LICENSE_READ_ONLY, PLAN_LIMIT_EXCEEDED, ACCOUNT_DISABLED), pagination, message keys ready for translation, all datetimes in UTC	1	A1	AIBD-037
P1-A3	CI on every push: Pint, Larastan, Pest with MySQL and Redis services	0.5	A1	
P1-A4	Shared OpenAPI 3.0 file published for the Vue and Flutter teams	0.5	A2	
P1-A5	Joint session with the AI team: settle the integration decisions and approve contract v1; write the decision record	1		AI-002, AI-003, AI-004

B Database and models

7 tasks · 8 days

P1-B1	Load the current PortSync SQL and base DDL fixes; generate draft migrations	1	A1	AIBD-038
P1-B2	Review migrations for access, marine, terminals, storage, yard and equipment (diagrams 2A, 2B)	1.5	B1	
P1-B3	Review migrations for request, BAPLIE, Part 1, Part 2, plan, execution and learning (2C, 2D, 3)	1.5	B1	AI-written tables
P1-B4	Real CHECK constraints, generated columns (`active_key`, `available_capacity`, `is_drifted`) and ENUMs	1	B2, B3	
P1-B5	Eloquent models and relations; one PHP enum per status column, SQL names only	1.5	B4	Shared vocabulary
P1-B6	Seeders: reference rows and a Damietta-scale demo port (8 terminals, 36 berths, pilots, tugs, shifts)	1	B5	Same fixture as AI-015
P1-B7	Central SaaS database: separate connection, migrations and models for the 8 tables, no logic	0.5	A1	

C Authentication

5 tasks · 3.25 days

P1-C1	Web sign-in with Sanctum SPA (CSRF cookie, 422 / 401 / 403); profile with roles and terminal scope	1	B2	
P1-C2	`must_change_password` gate: only change-password and logout until cleared	0.5	C1	
P1-C3	Mobile sign-in: device id, platform, app version, 426 below `min_app_version`, 30-day sliding token, `user_device` row	1	B2	
P1-C4	Logout: end the web session; set `revoked_at` on the mobile token	0.5	C1, C3	
P1-C5	GET `/system/info`: port id and name, licence mode, token warning days, minimum app version	0.25	B2	

PHASE 2

Access control and master data

18 tasks · 16.25 days

GOAL

Every later endpoint inherits the same write chain, and the port's master data is complete enough for the AI to read.

EXIT CHECK

Using only the API, a Port Admin creates users and roles and sets up terminals, berths, tugs, pilots, yard and safety limits; a READ_ONLY licence state blocks all of it with 423.

D Access control and audit

6 tasks · 5.25 days

P2-D1	Role policies for the 12 roles, terminal-scope middleware (403); field users see only their own tasks	1.5		
P2-D2	Write chain in fixed order: auth, password gate, scope, app version, licence mode, version token, rules, plan limits	1	D1	
P2-D3	Audit service: write and audit row in one transaction; refused writes as REJECTED_ATTEMPT; `actor_id` NULL for system	1		AIBD-025
P2-D4	Version token on `updated_at`; a stale write returns 409	0.5	D2	
P2-D5	Licence guard on `license_state`: READ_ONLY and SETUP allowlists as route tags (423); plan-limit service (422)	1	D2	AIBD-023
P2-D6	GET `/license/state` for the banner	0.25	D5	

E Users and roles

3 tasks · 2.25 days

P2-E1	Create user with one-time password (plan limit); deactivate under a row lock (409 for self or last Port Admin)	1	D1-D3	
P2-E2	Assign role with unit check; end role with `active_to`	0.75	E1	
P2-E3	List a user's devices; revoke one device or all tokens	0.5	E1	

F Master data

9 tasks · 8.75 days

P2-F1	Port profile, navigation channel (depth on chart datum), waypoints, typed `port_setting` editor	1	D1-D3	AIBD-016
P2-F2	Terminals (creates the TERMINAL unit), berths (`max_vessel_loa` within length), `berth_terminal` (at least one per berth)	1.5	D1-D3	AIBD-016
P2-F3	Tugs, tug masters (one per tug), pilots, rank limits, shifts (no overlap), pilot schedules	2	D1-D3	AIBD-016
P2-F4	CT equipment and capabilities; yard zones, blocks and slots with reefer, DG and OOG details	1.5	D1-D3	Yard input
P2-F5	Safety constraints (no overlap for same type and scope); tide constituent set activation with datum check	1	D1-D3	AIBD-016, AIBD-040
P2-F6	Terminal productivity profiles (no overlapping periods)	0.5	F2	AIBD-016
P2-F7	ResourceChanged domain event on berth, tug, pilot and equipment changes (listener in Phase 4)	0.5	F2, F3	AIBD-004
P2-F8	Audit log browsing; master-data history as a filter on it	0.5	D3	
P2-F9	Seed the AI `port_setting` keys and AI reference rows from the AI-009 spec	0.25		AIBD-021, AIBD-036

PHASE 3

Requests, BAPLIE and the AI bridge

20 tasks · 15.25 days

GOAL

The first AI-triggering flows exist and a mock worker answers them through the agreed contract. Work package I gates Phases 4 to 6.

EXIT CHECK

Customs approval creates a QUEUED run, the mock writes windows, the completion message arrives and the dashboard receives the SSE event; a BAPLIE upload does the same through `queue:edi`.

G Berthing requests

4 tasks · 3.5 days

P3-G1	Create request: vessel found or created by IMO (check digit), per-call particulars (drafts, block coefficient, GT, DWT), cargo lines	1		Part 1 inputs
P3-G2	SM-4 transition service: status, history row and audit in one transaction; the only code that writes `berthing_request.status`	1	G1	AIBD-011
P3-G3	Edit and cancel (409 after approval), preliminary approval (net tonnage and moulded depth required), return, resubmit, reject	1	G2	
P3-G4	List and search by status, vessel, terminal and ETA	0.5	G1	

H BAPLIE

8 tasks · 3.75 days

P3-H1	Upload to the shared volume, SHA-256, duplicate rule on `live_hash_key` (409), row STAGED / PENDING	1	G1	AIBD-015, AIBD-030
P3-H2	Auto-match by IMO and ETA within 24 h (422 on zero or several matches)	0.5	H1	
P3-H3	Enqueue the C-04 parse job	0.25	H1, I1	AIBD-007
P3-H4	Diagnostics and parse result API; manifest data hidden from other terminals	0.5	H3	AIBD-024
P3-H5	Withdraw (409 if a confirmed plan used the file; latest REPLACED becomes ACTIVE)	0.5	H1	
P3-H6	Correct staged data and re-validate	0.5	H4	BLOCKED D-03
P3-H7	Edit vessel particulars	0.25		
P3-H8	Manifest completeness gate: customs approval refused while any line lacks full or empty	0.25	H4	AIBD-035

I AI bridge and mock

8 tasks · 8 days

P3-I1	Job publisher: JSON envelope v1 pushed with RPUSH on an unprefixed Redis connection; builders for C-01 to C-05	1		AIBD-001 to 007
P3-I2	Run row first: the run row is created in the same transaction as the status change; push after commit; sweeper re-enqueues old QUEUED rows	1	I1	AIBD-012 to 015
P3-I3	Completion consumer: supervised artisan command on `queue:ai-results`, one handler per job type; poller safety net; stale RUNNING runs shown as failed with retry	1.5	I1	AIBD-010, AIBD-039
P3-I4	SSE endpoint for the dashboard, events filtered by role and terminal, within 5 s of commit	1.5		BLOCKED D-06
P3-I5	Grants script: one database account per worker; `audit_log` INSERT and SELECT only	0.5		AIBD-028
P3-I6	Mock workers: consume each queue, write contract-shaped rows including failure outcomes, send completion messages	1.5	I1	Replaced in Phase 6
P3-I7	Customs approval: CUSTOMS_APPROVED, part1_run QUEUED, C-01, answer 202	0.5	G2, I2, H8	AIBD-001
P3-I8	Contract tests in our CI against the JSON Schemas of contract v1	0.5	I1	AI-080

PHASE 4

The core loop

15 tasks · 13.25 days

GOAL

A request goes from customs approval to a confirmed plan. J, K and L run in parallel from seeded rows and the mock, then join at the end of the phase.

EXIT CHECK

On the mock, one request goes customs, window confirm, recommendation, override, plan confirm; the 409 path works when a tug is taken between review and confirm.

J Entry window

6 tasks · 4.5 days

P4-J1	Part 1 result API: latest run, outcome, windows, blocked periods with limiting constraint, data sources and ages	0.75		AIBD-026
P4-J2	Completion handler for PART1_RUN: SSE and ENTRY_WINDOW_READY; FAILED shown with a recompute action	0.5	J1	AIBD-022
P4-J3	Confirm a window: latest result and newest usable forecast (409), proposal CONFIRMED, request ENTRY_WINDOW_CONFIRMED, `solver_run` QUEUED, C-02 NEW_REQUEST	1	J1	AIBD-002, 013, 034
P4-J4	Reject all (allowed in READ_ONLY), recompute, reopen (409 if a plan is confirmed)	0.75	J1	C-01
P4-J5	Manual environmental data: completeness and horizon check (422), `forecast_run` MANUAL with points, audit, C-01 MANUAL_DATA	1		AIBD-009
P4-J6	Current conditions view with surge, source, age and stale warning	0.5		AIBD-033

K Recommendation review

5 tasks · 4.5 days

P4-K1	Completion handler for PART2_SOLVE: RECOMMENDATION_GENERATED with history and audit; NO_FEASIBLE_PLAN keeps ENTRY_WINDOW_CONFIRMED; RECOMMENDATION_READY	0.75		AIBD-011
P4-K2	Recommendation API: current version, options, tugs, rationale from template and factors, FIFO marked fallback, binding constraints from `result_detail`; records `recommendation_review`	1	K1	AIBD-026
P4-K3	Regenerate before confirmation (409 after, 423 in READ_ONLY), C-02 MANUAL	0.5	K1	AIBD-003
P4-K4	Override one component with a reason: validate, new MODIFIED_BY_STAFF version, `recommendation_override` row, previous version REPLACED	1.5	K2	BLOCKED D-03
P4-K5	Resource change listener: find pending recommendations that use the resource, enqueue C-02 RESOURCE_CHANGE	0.75	K1, P2-F7	AIBD-004

L Plan confirm and cancel

4 tasks · 4.25 days

P4-L1	Confirm plan in one transaction: reviewed and current (409), lock and re-check berth, tugs and pilot, plan CONFIRMED, components RESERVED, version CONFIRMED, request PLAN_CONFIRMED; rollback and 409 on a clash	2		AIBD-017
P4-L2	Cancel before any start: release resources, PLAN_CANCELLED; if the call continues, back to ENTRY_WINDOW_CONFIRMED and a new C-02; released resources fire RESOURCE_CHANGE	1	L1	AIBD-002, 004
P4-L3	Port Admin safety exception after a start (allowed in READ_ONLY, audit SAFETY_EXCEPTION)	0.5	L2	
P4-L4	Unified port state view: berths, tugs, pilots, requests and plans together	0.75		

PHASE 5

Dispatch and field execution

15 tasks · 13 days

GOAL

A confirmed plan reaches the pilot and the tug master, and field events come back as actual status and deviations.

EXIT CHECK

A seeded pilot receives a task, acknowledges it and finishes it through sync calls; a late STARTED event creates a deviation and a re-plan proposal on the mock.

M Dispatch

3 tasks · 2.25 days

P5-M1	Propose a pilot, a tug master and a tug from shifts, schedules and RESERVED components	1		
P5-M2	Officer confirms or replaces: availability, rank against LOA, shift, conflicts (422 / 409 / 423)	1	M1	
P5-M3	Save the assignment with audit; the component becomes ASSIGNED	0.25	M2	

N Notifications and realtime

4 tasks · 3.5 days

P5-N1	Notification service: sends after commit, one `notification_log` row per send, channels WEB and MOBILE, all 12 types	1		AIBD-022
P5-N2	Mobile WebSocket through Reverb, private channel per user; the app fetches content by REST	1		
P5-N3	Re-send at T+10 and T+20, ACK_ESCALATION at T+30 to the officer and the manager; cancelled on acknowledgement	1	N1	
P5-N4	Pre-execution reminder at the `port_setting` lead time (default 2 h)	0.5	N1	

O Mobile sync and execution

8 tasks · 7.25 days

P5-O1	My tasks and GET / `sync/changes`	1	M3	
P5-O2	POST / `sync/events:` applied once by `client_event_id`, in order, a rejection reason per event, creation events 423 in READ_ONLY	1.5		
P5-O3	Acknowledgement: notification ACKNOWLEDGED, retries cancelled, the officer sees it by SSE	0.5	O2, N3	
P5-O4	Status updates through `status_type` (monotonic); plan and request COMPLETED when all parts are done	1	O2, M3	
P5-O5	Store `client_sent_at`; flag `clock_skew_suspect` above 5 minutes	0.25	O4	AIBD-018
P5-O6	Deviation detection into `deviation_log` with thresholds as `port_setting` keys per component type; browse and filter	1	O4	AIBD-020
P5-O7	Significant delay: C-02 REPLAN (notify only in READ_ONLY); confirm or discard the proposal in one transaction, STARTED components kept	1.5	O6	AIBD-005
P5-O8	Terminal supervisor status from the tablet, same deviation path	0.5	O4	

PHASE 6

Yard, port licence and real AI

13 tasks · 11.5 days

GOAL

The last port-side features land and the mock is swapped for each real worker the AI team delivers, EDI Parser first.

EXIT CHECK

Every workflow whose worker has been delivered passes end to end on the real worker; the rest pass on the mock and are listed by name.

P Yard and TOS hand-off

4 tasks · 2.75 days

P6-P1	On plan confirm for a container vessel with an ACTIVE BAPLIE: `terminal_recommendation` YARD_REC_REQUESTED and C-03; in READ_ONLY record the reason and notify the planner	0.75		AIBD-006, 014
P6-P2	Yard result API for the owning terminal only; YARD_RECOMMENDATION_READY or _FAILED	0.5	P1	AIBD-024
P6-P3	Accept (no reservation), adjust (re-validated), reject (allowed in READ_ONLY)	1	P2	BLOCKED D-03
P6-P4	TOS-ready view; record EXPORTED_TO_TOS (allowed in READ_ONLY)	0.5	P3	

Q Port licence

3 tasks · 2.5 days

P6-Q1	Two-stage mode decision (port status, then licence) with every read-only reason; scheduled AI jobs keep running in READ_ONLY	0.75		AIBD-023
P6-Q2	Check-in client: random nightly run and Check In Now; Ed25519 verification; iat ordering; clock rollback guard on `max_seen_time`; tested against a stub until Phase 7	1.5	Q1	
P6-Q3	Licence status page data for the Port Admin	0.25	Q1	

R Real AI and tests

6 tasks · 6.25 days

P6-R1	Swap the mock for each delivered worker (EDI, then Part 1, then Part 2); contract tests green on both sides	1.5		AI-080
P6-R2	Evaluation and drift data for the Port Admin; DRIFT_ALERT from Part 3 completion messages	0.75		AIBD-022
P6-R3	End-to-end workflows WF-1 to WF-8 with the AI team, SSE delay measured	2	R1	AI-081
P6-R4	Failure paths: worker down, stale run sweep, solver timeout to FIFO, 409 race on plan confirm, Redis down at commit	1	R1	AIBD-039, AI-082
P6-R5	Performance: API p95, SSE within 5 s, solver path within 12 s end to end	0.5	R3	NFR-PERF-1, 3, 4
P6-R6	Prototype server: model volume, BAPLIE volume, supervisor for workers and the completion consumer	0.5		AIBD-030, 032

PHASE 7

Central SaaS

8 tasks · 8.25 days

GOAL

The central SaaS runs its own logic, so a port checks in against a real licence.

EXIT CHECK

A port registered in the SaaS gets a licence, checks in and receives a token; suspending the licence drops the port to READ_ONLY at its next check-in.

S SaaS core

6 tasks · 6.5 days

P7-S1	SaaS admin sign-in with email one-time code (cache store), logical roles, self and last-admin lock	1.5	
P7-S2	Licence plans and governorates	0.5	S1
P7-S3	Port registry: public id, API secret shown once (hash stored), rotation, lifecycle with recorded readiness confirmation	1.5	S1
P7-S4	Licences: issue, early renewal as PENDING, suspend with `on_hold`, reinstate, revoke; daily expire-then-activate job	1.5	S3
P7-S5	Check-in API with lazy Ed25519 signing and 14-day refresh; replaces the Phase 6 stub	1	S4
P7-S6	Platform audit in the same transaction as every write, refused attempts included	0.5	S1

T Billing and monitoring

2 tasks · 1.75 days

P7-T1	Billing transaction per licence year: create (one live per period), paid with reference, failed, refund, cancel	1	S4
P7-T2	Fleet monitor; audit and billing history browsing	0.75	S5

BACKLOG

Post-MVP backlog and open inputs

These items are designed and some are already decided, but FRD v3.1 places them after the MVP. They do not appear in the phases.

Item	Scope	Effort
FCM wake-up push (FW-14)	PUT / `api/v1/devices/push-token` ; `push_token` on `user_device` cleared on logout, revoke, deactivation and invalid-token answers; content-free send for the default set; PUSH rows in `notification_log` ; `push_enabled` port setting. Schema change already decided (FCM SQL change report).	1.5
Manual licence token	SaaS generates a token file for an ACTIVE licence; the port uploads it, including in READ_ONLY.	1
Signing-key rotation	Prepare, activate and retire steps for the Ed25519 signing key.	1
Bilingual error messages (FW-11)	Arabic and English texts for every error code; the message keys already exist from P1-A2.	1
Login lockout	Lock after repeated failed sign-ins; failures are already audited.	0.5

OPEN INPUTS

When	What is needed
Before Phase 1	The current PortSync SQL and the base DDL fixes, so P1-B1 generates the right tables.
Before Phase 1	A start date, so each phase gets real dates and lines up with the Vue and Flutter plans.
Before Phase 1	Where the SaaS API lives: inside project-backend as a second app, or a fourth repository.
Phase 1	A target delivery point from the AI team for each worker (EDI, Part 1, Part 2, Part 3).
Phase 2	The AI-009 seed specification: constraint types, objective terms, rationale templates, model keys.
Phase 3	The licence signing key for the prototype: a local key file or a KMS (P6-Q2 and P7-S5 depend on it).
Any time	Who keeps the OpenAPI file in sync with the Vue and Flutter teams.

APPENDIX

Appendix — AI dependency coverage

Every backend dependency in the AI team's file maps to a task, except AIBD-027 (data-gated future work) and AIBD-031 (a firewall rule for Port IT).

AIBD	What the AI team needs	Backend tasks
001	Part 1 job on customs approval, recompute, manual data, reopen	P3-I1, P3-I7, P4-J4, P4-J5
002	Initial solve after window confirm, or cancel with the call continuing	P4-J3, P4-L2
003	Regenerate before confirmation	P4-K3
004	Resource-change re-solve, including released resources	P2-F7, P4-K5, P4-L2
005	Delay re-plan on a significant deviation	P5-O7
006	Yard job on plan confirm	P6-P1
007	BAPLIE parse job	P3-H3
008	Override, yard adjustment and BAPLIE correction validation	P4-K4, P6-P3, P3-H6
009	Manual environmental data, then recompute	P4-J5
010	Completion signal consumed, SSE pushed	P3-I3, P3-I4
011	Request transitions with history in one transaction	P3-G2, P4-K1
012-015	Run rows created before enqueue	P3-I2, P3-H1, P3-I7, P4-J3, P6-P1
016	Master data with its validations	P2-F1 to P2-F6
017	Reservations as occupancy truth; booking re-check	P4-L1
018	Actual status with device and server times, clock-skew flag	P5-O2, P5-O5
019	Override records with reasons	P4-K4
020	Deviation thresholds and significance rule	P5-O6
021	AI `port_setting` keys	P2-F9
022	Notifications for AI outcomes	P4-J2, P4-K1, P5-N1, P6-P2, P6-R2
023	Licence gating before enqueue; scheduled jobs in READ_ONLY	P2-D5, P5-O7, P6-Q1
024	Terminal isolation of AI outputs	P3-H4, P6-P2
025	Audit for state changes caused by AI results	P2-D3, P3-G2
026	Read endpoints for AI results	P4-J1, P4-J6, P4-K2, P6-P2, P6-R2
027	Model review fields (data-gated)	Not planned: future work
028	Database accounts per worker	P3-I5
029	Redis with the three queues	P1-A1, P3-I1
030	BAPLIE volume readable by the parser	P3-H1, P6-R6
031	Outbound HTTPS to weather providers	Port IT, not backend
032	Worker supervision and health	P6-R6
033	Same surge formula in the conditions view	P4-J6
034	Window re-check at confirmation	P4-J3
035	Manifest completeness gate	P3-H8
036	Seeders for AI reference data	P2-F9
037	UTC storage and port timezone	P1-A2
038	One owner for schema migrations	P1-B1 to P1-B4
039	Stale runs visible to staff, with retry	P3-I3, P6-R4
040	Tide set activation with datum check	P2-F5